ECE/CS 285 Advanced Image Synthesis

Homework 1 – Suggested Workflow

This is a staged implementation plan for HW1. As you work, check geometry and placement with unlit colors, but the scene is not considered finished until all features it uses are implemented and its result is visually comparable to the final reference (read the comment at the top of each scene). Use the exact scene selectors below, and run one scene at a time while debugging. All provided scenes, including the two full movie sequences and all Tier 3 cinematic scenes, are required, along with an original scene you create. The base requirements are mandatory; choices explicitly marked optional or extra credit in HW1 remain optional. During development, run individual scenes to isolate problems; use `reyes all` to generate final deliverables.

1. To begin, read Cook et al.'s REYES paper to understand the bulk of the pipeline and skim over the RenderMan v.3.2.1 spec (especially the primitive section, as you'll use it as a reference when implementing quadrics). You do not need to implement or understand every feature in the full RenderMan specification before you start. Furthermore, read HW1 thoroughly so that you know what you'll be doing, and in particular be aware of the AI policy.

2. Build and run the starter kit before changing it. Follow `README.md` for your chosen language and platform, and run the first test scene, `FirstPrimitive_sphere`. The shorthand `reyes <selector>` below means your built executable: `build/reyes <selector>` for a C++ single-configuration build, `.\build\Release\reyes.exe <selector>` for Windows Visual Studio, or `cargo run --release -- <selector>` from `rust/`. The unmodified kit will write a fully transparent black image. Rust also needs a C++ compiler for the shared scenes; C students should follow the `README`'s source-replacement and writer-adaptation instructions.

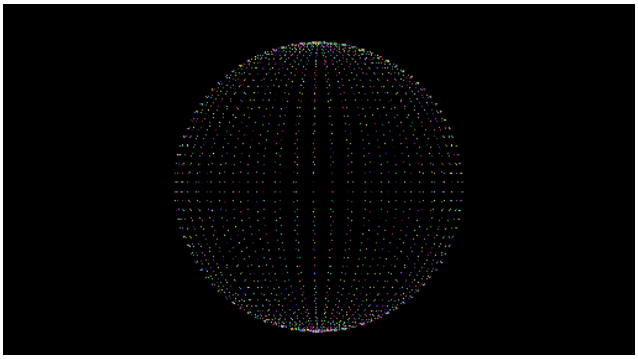
3. Implement graphics state and the transformation/projection conventions from handout Section 2. Include rotation, translation, scaling, concatenation and a nested transformation stack. Establish the session/frame defaults and restore frame state correctly. You can perform quick checks, e.g., starting from identity: `RiTranslate(1,2,3)` followed by `RiScale(2,3,4)` maps the point `(1,1,1)` to `(3,5,7)`. Double check your machinery as you build it to ensure correctness. You can also test `RiConcatTransform()` with an equivalent matrix as well.

4. Save the pre-world camera transform when `RiWorldBegin()` starts object placement. Map a few known 3D points to raster coordinates, with the eye looking along `+z`, `+y` up in camera space and row zero at the top of the image. Store them in a simple framebuffer and use the supplied writer. Later, the scenes `OrthoVsPersp_ortho` and `OrthoVsPersp_persp` check both projections once you have spheres, cylinders and lighting. For these debug splats, put the chosen RGB and alpha `1` at each marked pixel, leave unmarked pixels at RGBA zero, and pass four floats per pixel to the writer.

5. At this point, you're ready to work on the micropolygon grids. Create a rectangular grid data structure, recalling that an `n × n` grid of micropolygons has `(n + 1) × (n + 1)` vertices. Store all the necessary info at each vertex. Keep the grid indices distinct from the shader's `u,v` values, because on meshes the latter can be supplied texture coordinates. Start with a fixed grid resolution; choose it from projected bounds in step 11.

6. Now that you have the micropolygon grids working, implement one primitive by evaluating its parameterization over the grid. I suggest starting with `RiSphere`, following the RenderMan spec and the corrections in handout Sec. 3. Construct the sphere around the origin in object space since the current transformation places it in the scene. Implement all the behavior for `radius`, `zmin`, `zmax` and `thetamax`, including sphere-only clamping of `zmin`/`zmax`. Start with a complete sphere and add partial-sphere checks after the full sphere works correctly.

7. Transform the grid vertices into camera space, then project them to raster space. Temporarily splat the vertices into pixels to check the shape and placement. Use `FirstPrimitive_sphere`, a sphere five units in front of the eye. Before the shader system exists, use `RiColor` directly; once it exists, `CONSTANT` must produce the same unlit color. This point image is only for debugging, so gaps between points are expected and it should look something like this:



8. Sample the micropolygons. For now, use a single sample at the center of each pixel. You will implement multiple jittered samples per pixel in Step 10. Compute each micropolygon's bounding rectangle in raster pixels, intersect it with the image bounds, and inspect only the samples in the pixels it touches. Split each quad into two triangles. Test coverage with triangle edge functions or barycentric coordinates, skip zero-area triangles, and use a consistent edge-ownership rule so a shared diagonal or edge does not generate duplicate hits. Initially use one corner's color for a covered sample; you'll add interpolation later in step 16. Interpolate camera-space depth now, even while color is temporarily flat, so the z-buffer compares depth at the sample location. Handout Section 4 specifies the depth/interpolation convention. Do not perform a perspective divide on a micropolygon that fails the eye-plane rule in Section 3.

9. Implement an opaque z-buffer at each sample. Start with no hit and depth +infinity. For every covered sample, retain the nearest hit. At image output, average the sample colors and opacities with a box filter; uncovered samples contribute zero color and zero opacity. Hand the supplied writer a top-row-first buffer of floats, four per pixel: the premultiplied color and alpha, exactly as handout Section 4 describes. The writer converts and clamps them; do not do those conversions a second time. `FirstPrimitive_sphere` should now be a filled silhouette. Shading is still allowed to be temporarily unlit at this milestone.

10. Extend the framebuffer to `m × n` sample locations per pixel, from `RiPixelSamples`. Divide each pixel into m × n equal cells and choose one random position inside each cell. Generate these positions once for the current image and reuse them for every primitive; choose offsets independently for every sample in every pixel. A fixed random seed is useful for repeatable debugging. Once the torus and `CHECKERBOARD`/`STANDARD` shaders work

(steps 12 and 15), `AliasingTest_1x1` and `AliasingTest_4x4` check the sampling behavior. `OrthoVsPersp_persp` also uses 2 × 2 samples; after its cylinder exists in step 12, its silhouettes provide another sampling check. `FirstPrimitive_sphere` requests 1 × 1, so it should still look less smooth.

11. Implement bounding and dicing from handout Section 3: project the primitive's bound, measure its diagonal in pixels, choose and clamp the grid resolution, and apply the special rule for bounds crossing the eye plane. Reuse the same grid evaluator at the chosen resolution. Check large and small shapes with `DiceRateTest` after the sphere and torus work; the final shaded reference also requires `STANDARD`. Coarser grids change geometry and shading, but do not by themselves explain holes in an otherwise connected surface. If a filled grid develops cracks, also inspect indexing, coverage tests and edge ownership. At this point, your `RiSphere` should be fully working except for shading.

12. Implement `RiCylinder`, `RiCone` and `RiTorus` with their partial-primitive parameters. Use `FirstPrimitive_cylinder`, `FirstPrimitive_cone` and `FirstPrimitive_torus` to inspect each shape. Then check geometry/placement in both `OrthoVsPersp` variants and `SimpleAnimation` (which renders frame 75 only). These scenes can be inspected with temporary unlit colors; their final reference appearance requires `STANDARD`. `QuadricZoo` is the final combined quadric check after lighting and transparency work. Check `Tunnel` for geometry placement and transform save/restore, and `TransparencyTest` for partial quadrics; their final checkerboard and transparency effects arrive in steps 15 and 17.

13. Implement `RiPointsPolygons` using the same per-primitive pipeline. Start with an unlit quad, then a triangle whose final grid row collapses to one vertex, then a fan for polygons with more than four vertices. Use the same interpolation weights for position, supplied normals and `st`. Bound, dice, shade, bust and free one polygon before advancing to the next; do not retain a mesh's grids. A simple temporary unlit scene is enough to check geometry now. Run the complete `PolygonTest` after textures, lighting and `BUMPY` are available; its quad/two-triangle comparison checks matching texture mapping, with small sampling differences allowed. The floors in `ShadowTest` and `BumpVsDisplacement` use tiled polygons. `Gumbo`, `McQueen`, `Buzz` and `WallE` exercise larger meshes later; supplied normals and, where present, `st` must reach their shaders.

14. Implement `RiSurface()` and `RiDisplacement()` to bind arbitrary callbacks and copy their parameter blocks, then build the shader context at every grid vertex. Respect the parameter lifetime, alignment and callback re-entry guidance in handout Section 5. Compute analytic quadric normals and polygon face/interpolated normals, and transform normals by the inverse transpose. Preserve unflipped normals for displacement. When displacement is bound, run it over the whole grid, then compute finite-difference tangents and recompute normals from the displaced grid; finally run the surface shader over the whole grid. Use forward differences, with backward differences on the last row and column. Identify degenerate rows from the undisplaced grid so poles, apexes and collapsed rows have a stable fallback. Use the geometric normal to decide the eye-facing orientation for lighting as described in the handout. These are passes over the grid, not both shaders called consecutively at each vertex.

15. Implement the required shaders and textures in the order below. Use small opaque scenes of your own to isolate a feature when a supplied scene still depends on later work. At this stage

you can check lighting and patterns, but the final image also needs the color/opacity interpolation in step 16 and, where applicable, transparency in step 17.

Lighting: Implement `CONSTANT`, then `STANDARD`. `FirstPrimitive_sphere` must keep its unlit appearance under `CONSTANT`. Check lighting in both `OrthoVsPersp` variants, `DiceRateTest` and `SimpleAnimation` (frame 75). The interiors of `QuadricZoo` must be lit; its transparency comes later.

Checkerboard: Implement `CHECKERBOARD` as a dispatcher to arbitrary subshaders, with the parameter/input restoration rules in Section 5. Check patterns in `AliasingTest_1x1`, `AliasingTest_4x4` and `Tunnel`.

Textures: Implement `RiMakeTexture()` and `RiTexture()`, then use `UVTest` to check orientation and lookup conventions. Use the Rust `image` crate or the supplied C/C++ image loader, as appropriate.

Displacement: Implement the specified `perlinNoise()` and `BUMPY`, and check `DisplacementTest`, then `Tornado` and `Rocket` for geometry. `BumpVsDisplacement` supplies its own comparison shaders: the two spheres should show similar bump shading, but only the displaced sphere has a lumpy silhouette. Its white floor has no shadows until step 18. Return to `PolygonTest` now that textures, vertex normals and displacement are available. `Earth`, `Tornado`, `ShaderTest` and parts of `Rocket` also need step 17's transparency before their complete reference appearance is expected.

16. Interpolate **`Ci`** and **`Oi`** from the selected triangle's grid vertices at each covered sample, using the convention in handout Section 4. Preserve an opacity of exactly one for fully opaque triangles (or use the small tolerance allowed in Section 7). The surface shader runs at the grid vertices, not at every visibility sample. Recheck silhouettes, texture transitions and the quad/two-triangle comparison in `PolygonTest`. Independently jittered renders need not match reference pixels exactly. After interpolation works, the `OrthoVsPersp` variants, `DiceRateTest`, both `AliasingTest` selectors, `Tunnel` and `SimpleAnimation` should have all their required features. Check their final appearance now.

17. Extend the per-sample z-buffer to a depth-sorted list of `(z, Ci, Oi)` hits. Insert hits in depth order, discard anything hidden behind an already stored fully opaque hit, and discard farther hits when a new opaque hit is inserted. Keep any nearer transparent hits. Composite the surviving hits from front to back using the per-channel equations in Section 7 of the handout, starting with zero accumulated color and opacity. Stop once the accumulated opacity reaches 1 in all three channels. Then filter the samples as before. Check `TransparencyTest` and `TransparencyOrder`, then return to `QuadricZoo`, `Earth`, `Tornado`, `ShaderTest`, `Rocket`, and `Explorer`'s transparent rings. Free the lists when the image is finished so frames do not accumulate memory.

18. Implement depth output. When `RiDisplay()` has type `"zfile"` or mode `"z"`, choose at each pixel the smallest camera-space `z` over its sample hits whose mean opacity is at least `0.5`; use `+infinity` if none qualifies. Call the supplied `writeDepth()` (`write_depth()` in Rust). Your file must be complete before `RiWorldEnd()` returns, because the scene loads it immediately. Render `ShadowTest` and use its source comment to diagnose wrong shadows. `BumpVsDisplacement` should now gain its floor shadows; `Gumbo`, `AndysRoom` and `WallEMovie` also rely on a depth pass. The shadow lookup itself is provided.

For orthographic depth, first make a small test scene of your own: put an opaque quad entirely inside a square orthographic screen window at a known positive camera-space depth. Interior covered pixels should store that depth, and untouched pixels should store +infinity. If you adapt `ShadowTest` for a shadowed orthographic test, also pass `0` for the FOV to `zfile_load()` and use the same camera transform and any camera scale as the depth pass; changing `RiProjection()` alone is insufficient. Then check the required `RadiatorSprings` scene, which exercises orthographic depth at larger scale.

19. Run the individual test-scene selectors and compare with their reference images. At this stage the dependencies should all be present. Revisit `Tunnel` for transform placement/restoration, and `Explorer` for nested transforms and eye-plane behavior, and check that rendering a scene after another scene preserves the same geometry and appearance as rendering it alone (allowing differences from random jitter). Avoid `reyes all` or the movie-containing tier batches during early debugging; the full sequences add time without isolating a new bug. A visual mismatch should be traced to the feature the scene tests, not corrected with scene-specific behavior in the renderer.

20. Render the larger mesh scenes, starting with `McQueen`. For local debugging, temporarily use 320 × 240 and 1 × 1 samples, and restore the supplied settings before the final run. The supplied scene and `obj.h` load the car for you. Keep the loop bound, dice, shade, bust, free, one polygon at a time, so grid memory stays bounded. Profile an optimized build if it is slow: examine how many pixels/samples each micropolygon visits and avoid allocating or retaining unnecessary objects in that loop. Once the picture works, use the scene's final 960 × 720 and 4 × 4 settings. Then check `Buzz` and `WallE`; `WallE` is particularly useful for matrix concatenation and transform restoration. Then render `Gumbo`, with two 25,500-triangle elephants, vertex normals, procedural wood built on your `perlinNoise()`, and depth-map shadows.

21. Once the individual features and larger mesh scenes are working, render all three tiers, including both full movie sequences. Use `reyes test`, `reyes intermediate`, and `reyes cinematic` to run the tiers separately, or `reyes all` to render everything. Compare the outputs with the supplied reference renders at the required resolutions and sample counts. Allow sufficient time for the cinematic scenes and full movie sequences, which can take considerably longer than the individual tests. Restore the original scene settings before your final runs, and investigate any discrepancies rather than introducing scene-specific behavior into your renderer. Tier 3 consists of `RadiatorSprings`, `AndysRoom` and `WallEMovie`. The movie has 72 frames, each with its own depth pass; use your measured frame times to budget the complete run.

22. Create one original scene that makes effective use of your renderer. A carefully composed still is sufficient. You may use the supplied models or documented external models and may use AI for the artistic scene work allowed by the handout. Keep the scene small enough to iterate on; a polished arrangement of a few assets can demonstrate the renderer well. Create its `.cpp` file directly in `scenes/`, add it to `CMakeLists.txt` for C/C++ (Rust discovers this location), declare its entry point and register it in `scenes/0_scene_table.cpp`, then include its final render with your submission.

23. Once all the required scenes are working, use the `ffmpeg` instructions in Section 8 of HW1 to assemble `SimpleAnimationMovie` (100 PNG frames) and `WallEMovie` (72 JPEG frames) into the required MP4 files. If you have extra time, attempt any additional features from the Extra Credit section. Document all extra credit and AI use in your `README`. Finally,

rebuild using the documented command and verify that all required scenes, including your original scene, have been rendered with the final version of your renderer. If you have already generated those outputs, you do not need to render them again. Make sure all required images, movie frames, and MP4 files are present, then package your submission as described in the handout.

Suggested two-week milestones:

- Days 1–2: Build/state/point projection
- Days 3–4: Opaque sphere and hider
- Days 5–6: Quadrics/dicing/lighting
- Days 7–8: Textures/displacement/meshes
- Days 9–10: Transparency/depth
- Days 11–12: Tier 2 integration tests; begin Tier 3 scenes and full movie renders
- Day 13: Original scene; continue required final renders
- Day 14: Final runs and packaging

If you have not reached the opaque-sphere milestone by the end of day 4, ask for help immediately. This plan assumes the graphics and programming prerequisites; leave extra time if you are learning either alongside the assignment.

Please pace yourself through this assignment so that you finish within the timeline. Do not leave things to the last minute! Also, after you are finished let me know if this workflow was helpful to you or if you have suggestions on how to improve it and make it easier to follow.

Best of luck!